CS 6158 Midterm Progress Report

Budget-Constrained Selective Runtime Monitoring

Shiyun Cheng Gengkun Shao

October 19, 2026

1 Research Goal and Progress

Our project asks how to allocate a fixed budget among runtime monitor-creation locations using historical execution frequency, cost, useful traces, violation reports, and redundancy. The primary research question is whether budget-aware selection preserves more violations than simple heuristics or *Valg*, an adaptive online monitor-creation policy, at comparable *realized* overhead. We originally proposed evaluation on held-out executions, including unique-trace and violation recall, runtime and memory, with software evolution and exploration as extensions.

At midterm, we have **implemented five selection baselines and a reproducible evaluation pipeline**, and performed **offline selection experiments on historical TraceMOP monitor-profile data for two real Java projects**. In parallel, we reproduced a subset of the official Valg artifact on four Java projects. The two studies serve different purposes: the offline experiment evaluates budget-allocation rules under a historical proxy cost, whereas the artifact reproduction validates the executable monitoring infrastructure and establishes preliminary realized-time baselines. Our budgeted policies have not yet been deployed inside Valg or compared with it at matched measured cost.

2 Implemented Modules and Methods

The Python selection pipeline reads per-group historical information and produces selected group IDs, budget usage, recall metrics, CSV tables, and plots. Five strategies are implemented: **random** feasible selection (30 draws per budget in the real-trace evaluation); **frequency ranking**; **standalone utility/cost ranking**; **redundancy-aware greedy**, which maximizes incremental distinct-trace gain per unit cost; and **additive knapsack**, which maximizes the sum of standalone trace counts using a discretized dynamic program. The knapsack surrogate can double-count traces shared by several groups; it is not an exact optimum for distinct-union coverage.

For historical trace IDs T_i attached to a candidate group i, a selected set S, and a proxy cost c_i, the current offline experiment approximates

$$\max_{S} \quad \left|\bigcup_{i\in S} T_i\right| \quad \text{s.t.} \quad \sum_{i\in S} c_i \le B, \qquad \text{TraceRecall}(S) = \frac{|\cup_{i\in S} T_i|}{|\cup_i T_i|}. \tag{1}$$

Here c_i is the workbook's `cost_proxy_event_checks`, *not execution time*. Candidate rows represent inferred location-by-specification groups; they are not yet verified as independently switchable monitor-creation sites. Real-trace violation weighting is not yet supported because the workbook lacks reliable location-attributed violation sets.

For the artifact study, we used the official Valg scripts in a prepared Docker environment and pinned project commits. Each Java-mode run executes the same project tests under no runtime verification (no-RV), ValgJ selective monitoring, and JavaMOP full monitoring. The trace-mode pair compares ValgT with TraceMOP. Raw logs, elapsed times, extracted violation counts, trace summaries, and the scripts used to aggregate the measurements are retained in the repository.

3 Experiments and Preliminary Results

3.1 Historical Real-Trace Offline Evaluation

We processed the partner-provided aggregation for `javadbf` (153 location-by-specification groups, 278 observed distinct trace IDs) and `jlemmagen` (66 groups, 2,115 trace IDs). Budgets were 10, 20, 30, 50, and 80% of summed event-check proxies. Random selection used 30 draws per budget. Table 1 reports the 20% result; the denominator is the union of trace IDs already present in the historical records.

Table 1: Offline unique-trace recall at a 20% event-check proxy budget.

Strategy	`javadbf`	`jlemmagen`
Random (30-draw mean)	86.51%	65.56%
Frequency	44.96%	88.84%
Utility/cost	97.84%	99.67%
Redundancy-aware greedy	97.84%	99.67%
Additive knapsack	89.57%	99.57%

The cost-aware approaches preserved most previously observed distinct traces at this proxy budget. Greedy and static utility/cost tied on both workloads, so the current results do not establish a benefit from explicitly modeling redundancy. Knapsack can perform worse because its additive objective rewards overlapping traces repeatedly. Frequency selections can be non-monotone because each budget independently scans a fixed ordering and admits any group that fits.

3.2 Partial Valg Artifact Reproduction

All four selected projects completed no-RV, ValgJ, and JavaMOP runs. Table 2 reports one end-to-end measurement per mode. The reduction is $1 - T_{\text{ValgJ}}/T_{\text{JavaMOP}}$; a negative value means that ValgJ was slower. Because each cell is a single run, the table establishes artifact execution rather than a statistically stable performance estimate.

Table 2: Java-mode subset reproduction: end-to-end time in seconds.

Project	no-RV	ValgJ	JavaMOP	Reduction
`xeger`	10.428	44.714	66.003	32.3%
`javadbf`	11.142	61.079	38.795	−57.4%
`route53-infima`	13.673	41.806	59.767	30.1%
`jlemmagen`	9.389	114.502	315.707	63.7%

ValgJ was faster on three of four projects, with a median reduction of 31.2%. Within every ValgJ/JavaMOP pair, the extracted logs contained the same number of distinct violation rows, although ValgJ produced fewer violation occurrences. We therefore report preservation of the observed violation signatures, not a general violation-recall claim.

Only `jlemmagen` completed both trace modes. As summarized in Table 3, ValgT reduced end-to-end time and duplicate trace instances while retaining most unique traces observed by TraceMOP.

Table 3: Trace-mode result for `jlemmagen`.

Metric	ValgT	TraceMOP	Relative result
End-to-end time (s)	184.600	349.056	47.1% reduction
Unique traces	1,840	2,115	87.0% retained
Duplicate instances	79,812	1,258,457	93.7% reduction

Figure 1 summarizes the two complementary parts of this artifact reproduction: realized Java-mode runtime and trace redundancy.

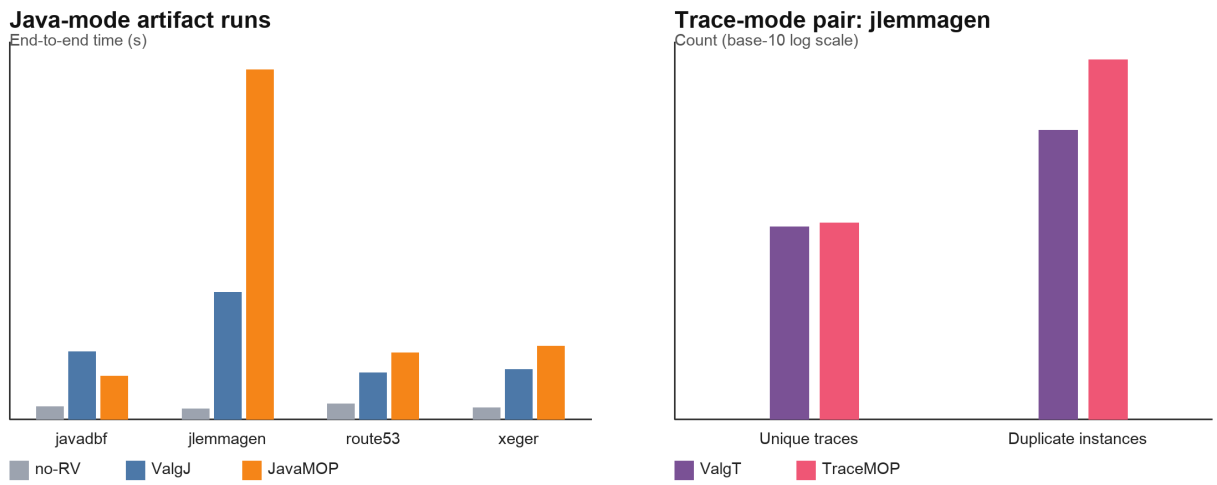


Figure 1: Subset reproduction: Java-mode end-to-end time and the one complete trace-mode pair. The trace counts are shown on a base-10 logarithmic scale.

The left panel compares end-to-end test time under no-RV, ValgJ selective monitoring, and JavaMOP full monitoring. ValgJ is faster than JavaMOP for `xeger`, `route53-infima`, and especially `jlemmagen`, but is slower for `javadbf`; therefore selective monitoring does not guarantee a speedup on every workload. The right panel isolates the intended mechanism on `jlemmagen`. Relative to TraceMOP, ValgT retains 87.0% of distinct traces while eliminating 93.7% of duplicate trace instances. This large reduction in repeated traces, together with a much smaller loss of unique behavior, is consistent with Valg's goal of avoiding redundant monitoring work. Nevertheless, the trace result covers only one project, and both panels report single runs, so the figure validates our setup rather than reproducing the paper's full evaluation or proving statistical performance gains.

4 Challenges, Risks, and What We Learned

First, **violation attribution is missing**: historical reports identify violations but do not show whether each proposed location would independently detect them. Second, **proxy cost is not realized overhead**: global instrumentation, shared monitoring work, and interactions between selected groups can invalidate an additive runtime interpretation. Third, inferred location-by-specification groups may not correspond to independently controllable creation-site switches. Fourth, the current real-trace data have no held-out evaluation split. Fifth, the artifact timings are single runs and only one project has a complete ValgT/TraceMOP pair. We therefore cannot yet claim generalization, memory savings, or superiority over Valg.

5 Schedule and Deliverables

We remain on track for the implemented selection-pipeline deliverable; a fully realized-cost-matched Valg comparison remains the main risk. **Oct. 19–25:** validate group IDs and extraction semantics, reproduce offline plots, and curate workloads. **Oct. 26–Nov. 1:** implement or test enable/disable control for selected monitor-creation sites and collect per-site violation associations. **Nov. 2–8:** profile representative configurations with repeated runtime and memory measurements. **Nov. 9–15:** compare budgeted policies and Valg at similar measured overhead using held-out executions. **Nov. 16–end of term:** complete robustness analysis, possible revision-transfer or exploration studies, the final repository, and the report.

Shiyun maintains the selection and offline-evaluation pipeline, while Gengkun coordinates Valg artifact reproduction, profiling, and runtime-integration work. If switchable monitors or incremental timing measurements prove infeasible, we will report a carefully labeled offline proxy-cost benchmark, investigate alternative selective instrumentation, and explicitly narrow our claims rather than treating proxy budgets as runtime savings.